

Desenvolvimento de algoritmo de reconhecimento de cores utilizando MATLABTM

Resumo

Este trabalho apresenta um algoritmo simples para reconhecimento de cores em imagens digitais. O programa foi feito em MATLAB e usa a conversão do espaço RGB para HSV. Depois disso, ele separa os canais de matiz, saturação e valor, cria uma máscara binária e conta quantos pixels estão dentro da faixa da cor procurada. O banco de dados foi montado com cinco imagens obtidas no Google Imagens e salvas como 1.jpeg até 5.jpeg. Para gerar uma visão mais completa do método, a mesma lógica foi executada para quatro cores: amarelo, azul, verde e vermelho. Os resultados mostraram que o método funciona melhor quando a cor é forte e o fundo é claro, mas também mostraram limites quando a cor fica perto de outra, como amarelo e laranja. Mesmo sendo um exemplo simples, o trabalho ajuda a entender ideias básicas de visão computacional que aparecem em apps de fotos, biometria facial, carros autônomos, recursos de acessibilidade e exames médicos.

1 Introdução

Reconhecimento de imagens é quando o computador tenta entender o que aparece em uma foto. Hoje isso é importante em muitas tarefas do dia a dia. No caso do Google Photos, por exemplo, o sistema pode detectar rostos, criar modelos numéricos desses rostos e juntar fotos com rostos muito parecidos no mesmo grupo [4]. No celular, o Face ID pode destravar o aparelho, confirmar compras e entrar em aplicativos com um olhar para a câmera [5]. Em carros autônomos, a percepção visual é usada para tarefas como detectar objetos no trânsito, achar a área por onde o carro pode andar e localizar faixas da pista [9].

Esse tipo de tecnologia também pode ajudar muita gente. O app Seeing AI foi criado para pessoas cegas ou com baixa visão e consegue ler textos, falar sobre objetos e dar descrições do que está perto [6]. Para pessoas daltônicas, também existem ferramentas que ajudam a identificar cores em tempo real por meio da câmera do aparelho [7]. Isso mostra como um sistema simples de cor já pode ter valor no dia a dia. Além disso, estudos de leitura labial e percepção áudio-visual mostram que a informação visual da boca pode

ajudar a entender melhor a fala, principalmente quando ela é usada junto com o som [8]. Na área da saúde, revisões recentes mostram que métodos de aprendizado profundo aplicados a imagens médicas podem ajudar na detecção de tumores e em outras tarefas de análise [11, 12]. Neste artigo, o foco não é reconhecer rostos ou tumores. A ideia foi fazer algo bem menor: distinguir cores em imagens. Mesmo assim, esse problema é útil para aprender uma etapa muito comum da visão computacional, que é separar partes de uma imagem a partir de alguma característica visual.

2 Objetivo

O objetivo deste trabalho foi criar um algoritmo simples de reconhecimento de imagens para distinguir cores. Mais exatamente, o programa deveria ler uma imagem, procurar uma cor escolhida e decidir se essa cor aparece em quantidade suficiente para ser considerada presente.

3 Metodologia

O algoritmo foi desenvolvido no MATLAB. O processo começa com a leitura da imagem por meio da função `imread`. Em seguida, a imagem é convertida de RGB para HSV com a função `rgb2hsv`, que transforma os valores de vermelho, verde e azul em matiz, saturação e valor [1]. Essa etapa é útil porque a matiz fica mais ligada ao “tipo de cor”, enquanto a saturação e o valor ajudam a eliminar partes muito apagadas ou muito escuras. Em um trabalho clássico sobre segmentação por cor, Sural, Qian e Pramanik mostraram que o uso de HSV pode identificar contornos de objetos de modo mais próximo da percepção humana do que o uso direto de RGB [10].

Depois da conversão, o código separa os três canais HSV. A seguir, ele usa limites fixos para cada cor. No caso do amarelo, o intervalo usado foi de 0,10 a 0,20 no canal H. Para azul, foi de 0,55 a 0,75. Para verde, foi de 0,25 a 0,45. Para vermelho, o tratamento foi diferente, porque essa cor aparece nas extremidades do círculo de matiz; por isso, a máscara considera valores menores que 0,05 ou maiores que 0,95. Em todos os casos, também foram usados os filtros $S > 0.4$ e $V > 0.4$. A máscara final é binária: pixel 1 quando a condição é verdadeira e pixel 0 quando ela é falsa.

Depois disso, o programa soma todos os pixels verdadeiros da máscara. Quando o total passa de 500 pixels, a imagem é marcada como positiva para a cor escolhida. Esse tipo de limiar fixo é simples e fácil de entender, mas a própria documentação do MATLAB mostra que a segmentação por cor nem sempre fica totalmente limpa em todos os casos, especialmente quando há variação de iluminação ou bordas com transição suave [2].

O banco de dados usado neste trabalho teve cinco imagens, obtidas por busca manual no Google Imagens e salvas como 1.jpeg, 2.jpeg, 3.jpeg, 4.jpeg e 5.jpeg.

Para montar a tabela completa de resultados, a mesma lógica do programa foi executada quatro vezes, uma vez para cada cor: amarelo, azul, verde e vermelho. É importante observar que, no Apêndice, o código original do autor foi mantido exatamente como foi enviado, inclusive com o laço `for i = 1:6`. Já para a análise deste artigo, foram consideradas as cinco imagens realmente disponíveis.

A Figura 1 mostra o conjunto de imagens usado no pequeno banco de dados.

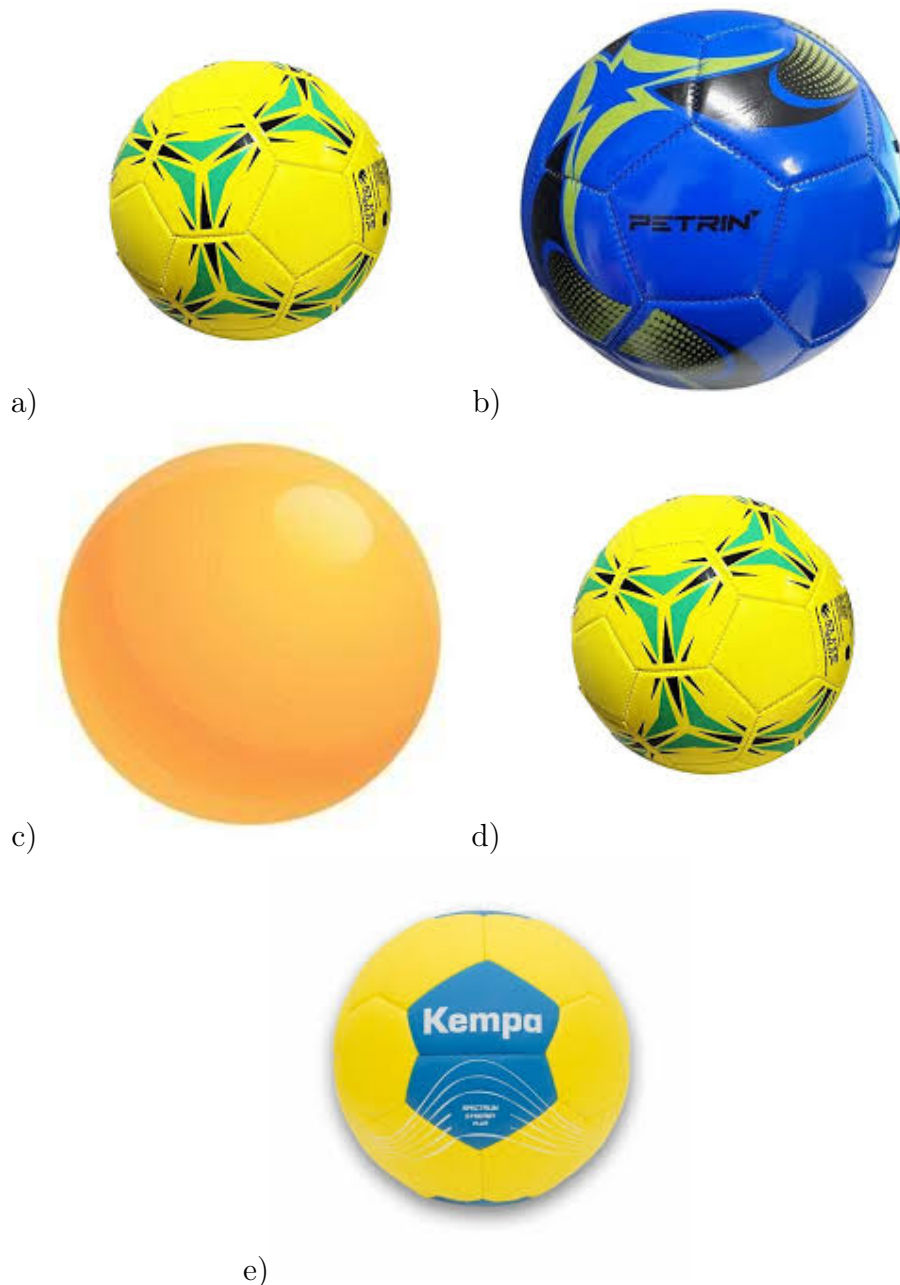


Figura 1: As cinco imagens usadas no banco de dados deste trabalho. a) 1.jpeg, b) 2.jpeg, c) 3.jpeg, d) 4.jpeg, e) 5.jpeg. Extraído de [4].

4 Resultados

A Tabela 1 apresenta a contagem de pixels detectados em cada máscara de cor. A regra de decisão foi a seguinte: se a cor teve mais de 500 pixels, ela foi considerada presente na imagem.

Tabela 1: Contagem de pixels por cor e decisão final usando limiar maior que 500 pixels.

Imagem	Amarelo	Azul	Verde	Vermelho	Cores detectadas
1.jpeg	13833	1	1645	0	amarelo, verde
2.jpeg	303	22329	28	0	azul
3.jpeg	11031	0	0	0	amarelo
4.jpeg	13833	1	1645	0	amarelo, verde
5.jpeg	13057	4725	23	0	amarelo, azul

Os resultados mostram que o método foi coerente nas imagens com cor forte e bem destacada. A imagem 2 foi marcada como azul, o que combina bem com a bola azul. A imagem 5 foi marcada como amarela e azul, o que também faz sentido, pois a bola tem regiões grandes dessas duas cores. As imagens 1 e 4 tiveram exatamente a mesma contagem porque os arquivos são iguais no banco de dados usado neste teste.

Também apareceu um limite importante. A imagem 3 foi detectada como amarela, mesmo parecendo mais próxima de amarelo-alaranjado. Isso mostra que um intervalo fixo de matiz pode confundir cores vizinhas. Então, o método funciona para exemplos simples, mas não decide tudo de forma perfeita.

Abaixo estão exemplos de saídas de texto do programa, usando o mesmo critério de decisão do código. Para fazer esse resumo completo, a lógica foi executada separadamente para cada cor.

Saída para cor = amarelo

Imagem 1 tem a cor escolhida

Imagem 3 tem a cor escolhida

Imagem 4 tem a cor escolhida

Imagem 5 tem a cor escolhida

Saída para cor = azul

Imagem 2 tem a cor escolhida

Imagem 5 tem a cor escolhida

Saída para cor = verde

Imagem 1 tem a cor escolhida

Imagem 4 tem a cor escolhida

Saída para cor = vermelho
(sem saída)

De forma geral, os resultados foram bons para um algoritmo curto e fácil de entender. Ao mesmo tempo, eles deixam claro que escolher faixas fixas funciona melhor em casos controlados. Mudanças de luz, sombra, brilho, reflexo e mistura entre cores próximas podem alterar a contagem de pixels e mudar a decisão final. Esse tipo de limitação também é compatível com a documentação do próprio MATLAB, que mostra que a segmentação simples por cor pode precisar de ajuste fino para ficar mais limpa [2].

5 Conclusão

Este trabalho apresentou um algoritmo simples de reconhecimento de cores em imagens usando MATLAB e o espaço HSV. O método foi baseado em três ideias fáceis de seguir: converter a imagem, criar uma máscara com limites de cor e contar quantos pixels passaram no filtro. Nos testes feitos com cinco imagens, o programa conseguiu identificar bem cores fortes, como azul, amarelo e verde, quando elas ocupavam uma parte grande da figura.

Mesmo com esse resultado positivo, o método ainda é simples e tem limites. Ele depende de faixas fixas e do valor de corte de 500 pixels. Por causa disso, pequenas mudanças de luz, sombra ou tom podem atrapalhar. Isso ficou claro no caso da imagem 3, que acabou entrando como amarela mesmo estando perto de laranja.

Como trabalho futuro, uma boa melhoria seria captar a imagem ao vivo com câmera, em vez de usar apenas um banco de dados salvo. A própria documentação do MATLAB mostra que é possível carregar imagens ao vivo a partir de uma webcam no fluxo de segmentação por cor [3]. Outra melhoria importante seria fazer o sistema procurar sozinho todas as cores presentes, sem que a pessoa precise dizer antes qual cor deve ser buscada.

Referências

- [1] MathWorks. *rgb2hsv – Convert RGB colors to HSV*. Documentação oficial do MATLAB. Disponível em: <https://www.mathworks.com/help/matlab/ref/rgb2hsv.html>. Acesso em: 10 maio 2026.
- [2] MathWorks. *Segment Image and Create Mask Using Color Thresholder*. Documentação oficial do Image Processing Toolbox. Disponível em: <https://www.mathworks.com/help/images/image-segmentation-using-the-color-thesholder-app.html>. Acesso em: 10 maio 2026.

- [3] MathWorks. *Acquire Live Images in Color Thresholder*. Documentação oficial do Image Processing Toolbox. Disponível em: <https://www.mathworks.com/help/images/acquire-live-images-in-the-color-thresholder-app.html>. Acesso em: 10 maio 2026.
- [4] Google Photos Help. *Set up & manage your face groups*. Disponível em: <https://support.google.com/photos/answer/6128838>. Acesso em: 10 maio 2026.
- [5] Apple Support. *Use Face ID on your iPhone or iPad Pro*. Disponível em: <https://support.apple.com/en-us/108411>. Acesso em: 10 maio 2026.
- [6] Microsoft Garage. *Seeing AI*. Disponível em: <https://www.microsoft.com/en-us/garage/wall-of-fame/seeing-ai/>. Acesso em: 10 maio 2026.
- [7] ColorADD. *ColorADD APP*. Disponível em: <https://www.coloradd.net/en/coloradd-app/>. Acesso em: 10 maio 2026.
- [8] SUMMERFIELD, Q. Lipreading and audio-visual speech perception. *Philosophical Transactions of the Royal Society B: Biological Sciences*, v. 335, n. 1273, p. 71–78, 1992. DOI: <https://doi.org/10.1098/rstb.1992.0009>.
- [9] WANG, H.; LI, J.; DONG, H. A Review of Vision-Based Multi-Task Perception Research Methods for Autonomous Vehicles. *Sensors*, v. 25, n. 8, p. 2611, 2025. DOI: <https://doi.org/10.3390/s25082611>.
- [10] SURAL, S.; QIAN, G.; PRAMANIK, S. Segmentation and Histogram Generation Using the HSV Color Space for Image Retrieval. In: *Proceedings of the International Conference on Image Processing*. 2002. DOI: <https://doi.org/10.1109/ICIP.2002.1038043>.
- [11] XUE, P.; WANG, J.; QIN, D. et al. Deep learning in image-based breast and cervical cancer detection: a systematic review and meta-analysis. *npj Digital Medicine*, v. 5, art. 19, 2022. DOI: <https://doi.org/10.1038/s41746-022-00559-z>.
- [12] DORFNER, F. J.; PATEL, J. B.; KALPATHY-CRAMER, J. et al. A review of deep learning for brain tumor analysis in MRI. *npj Precision Oncology*, v. 9, n. 1, p. 2, 2025. DOI: <https://doi.org/10.1038/s41698-024-00789-2>.

A Código original em MATLAB

A seguir está o código original enviado para o trabalho, agora com a formatação correta do $\text{\LaTeX}$. Ele foi mantido exatamente como foi fornecido.

```

clc;
clear all;

cor = 'vermelho';

if strcmp(cor, 'amarelo')
    H_min = 0.10;
    H_max = 0.20;

elseif strcmp(cor, 'azul')
    H_min = 0.55;
    H_max = 0.75;

elseif strcmp(cor, 'verde')
    H_min = 0.25;
    H_max = 0.45;

elseif strcmp(cor, 'vermelho')
    H_min = 0.05;
    H_max = 0.95;
end

for i = 1:6
    nome = sprintf('%d.jpeg', i);
    img = imread(nome);

    hsv = rgb2hsv(img);
    H = hsv(:, :, 1);
    S = hsv(:, :, 2);
    V = hsv(:, :, 3);

    if strcmp(cor, 'vermelho')
        mascara = (H < H_min | H > H_max) & (S > 0.4) & (V > 0.4);
    else
        mascara = (H > H_min & H < H_max) & (S > 0.4) & (V > 0.4);
    end

    quantidade = sum(mascara(:));

    if quantidade > 500
        fprintf('Imagem %d tem a cor escolhida\n', i);
    end
end

```

```
    end  
end
```